

QAMP Checkpoint 2 Report

Solving-Mean-Variance-Portfolio-Optimization-using-QAOA Public: Issue 28

December 14, 2025

Project Overview

Our project aims to develop a robust quantum optimization module for the Mean-Variance Portfolio Optimization (MVO) problem. We are targeting realistic constraints—specifically the cardinality constraint (selecting exactly K assets out of N), which is notoriously difficult for standard quantum algorithms to handle efficiently.

Our solution integrates four advanced algorithmic upgrades into a cohesive engine:

1. **ADAPT-QAOA:** To iteratively grow the ansatz and minimize circuit depth (CNOT counts) for NISQ hardware.
2. **Hard-Constraint XY-Mixers:** To confine the search strictly to the feasible K -hot subspace, avoiding invalid portfolio selections.
3. **Warm-Starting (WS-QAOA):** To bias the initial quantum state using classical approximations.
4. **CVaR Objective:** To optimize for the tail of the distribution, enhancing resilience against device noise.

Progress Made

Since the last checkpoint, we have successfully architected and implemented the core `quant_portfolio` Python stack. We have moved from theoretical design to a functional, modular codebase that generates, solves, and benchmarks portfolio instances.

Key accomplishments include:

- **Core Engine Implementation (`quant_portfolio`):** We have built a self-contained library that handles the end-to-end pipeline. This includes `data.py` for preprocessing returns (μ) and covariance (Σ), and `qaoa_core.py`, which implements the physics of the algorithm, including Dicke state initialization and Trotterized XY-mixers.
- **ADAPT-QAOA Logic:** We implemented the adaptive engine in `adapt_qaoa.py`. It currently supports a “pairwise” operator pool, allowing the algorithm to greedily select specific XY-swaps that maximize energy reduction at each layer, reporting precise gate counts.
- **Classical Baselines:** To ensure rigorous benchmarking, we implemented a classical solver module (`classical.py` and `mip.py`). This includes a Brute Force solver for small instances and a Mixed-Integer Linear Programming (MILP) adapter (using McCormick linearization) to establish the true global optimum for performance comparisons.
- **Benchmarking Infrastructure:** We developed `run_sweep.py`, a CLI tool that automates parameter sweeps across problem sizes (N, K), depths (p), and noise levels, consolidating results into CSV metrics for analysis.

Explorations and Experiments

Our primary exploration has focused on **Constraint Preservation**. We experimented with two distinct mixer strategies:

1. **Standard X-Mixer:** We implemented the standard R_x mixer as a baseline. As expected, we observed that while simpler to implement, it frequently explores invalid subspaces (portfolios with $\neq K$ assets), requiring heavy penalty terms. The requirement of the hyperparameter tuning in this stage is a constraint in itself.
2. **XY-Ring Mixer:** We successfully implemented a hard-constraint mixer using partial SWAP gates arranged in a ring topology. By initializing the system in a Dicke State (an equal superposition of all valid K -asset portfolios), we verified that the XY-mixer evolution keeps the probability mass entirely within the feasible subspace. This “hard constraint” approach effectively reduces the Hilbert space size, making the optimization more efficient.

Additionally, we explored **Warm-Starting**. Instead of initializing with a uniform superposition, we implemented a biasing routine based on asset expected returns. Preliminary tests suggest this “warm” initialization can steer the optimizer toward high-quality solutions faster than “cold” starts.

Key Insights and Learnings

- **The “Embedding” Challenge:** We learned that mapping constrained problems to qubits is non-trivial. While penalty terms are the standard textbook solution, they waste quantum resources. Our work confirmed that subspace restriction (via Dicke states and XY mixers) is a superior, albeit more complex, strategy for NISQ devices.
- **Classical Verification is Crucial:** Integrating the `pulp` MILP solver was a turning point. It provided a baseline that allowed us to debug our quantum energy calculations. We discovered that without a rigorous classical baseline, it is impossible to validate whether the quantum solution is truly “optimal” or just “good enough”.
- **Noise Sensitivity:** Early runs with the CVaR objective highlighted how quickly standard expectation values degrade under depolarizing noise. Shifting to CVaR (optimizing the best $\alpha\%$ of shots) drastically improved the apparent solution quality in noisy simulations.

Technical Challenges

- **Simulator Performance:** Simulating ADAPT-QAOA is computationally expensive because it requires calculating gradients for every candidate operator in the pool at every step. To overcome this, we adopted design patterns from the `QOKit` library, specifically the precomputation of diagonal energies, which accelerated our simulation loops significantly compared to naive matrix multiplication.
- **Dependency Management:** We faced challenges ensuring the MILP solver dependencies were portable. We mitigated this by implementing a fallback to a Brute Force solver in `classical.py` for smaller instances if the MILP solver is unavailable in the environment.

Current Status

The project is currently in the **Integration and Testing phase**. The internal stack is fully functional: we can run Standard QAOA and ADAPT-QAOA with XY mixers, apply warm-starting, and evaluate results using the CVaR objective. The code is structured into a reproducible pipeline (`quant_portfolio`), and we are generating CSV artifacts for mentor review. We have verified the correctness of the energy calculations against our classical MILP baseline.

Path Forward

Our plan to complete the MVP by January 30 is as follows:

1. **Advanced Formulation Testing (Weeks 1-2):** We will activate the code paths for Mean-Absolute Deviation (MAD) and Transaction Costs (MVO+TC). These are non-convex problems where classical solvers often struggle, making them ideal candidates for demonstrating quantum utility.
2. **Noise Model Refinement (Week 3):** We will upgrade our simple depolarizing noise model to include more realistic device-specific error channels (e.g., bit-flip, phase-flip) to stress-test the CVaR objective further.
3. **Final Benchmarking & Reporting (Week 4):** We will execute a comprehensive Sweep using `run_sweep.py` to generate the final plots comparing ADAPT-QAOA vs. Standard QAOA vs. Classical Solvers.
4. **Deliverables:** We will finalize the interactive Jupyter Notebook tutorial and the Qiskit community blog post, as outlined in our original roadmap.